

Mpango ERP 项目规范文档



1. 标准化AI通信协议.md

概述

基于Mpango ERP项目的多AI平台协调开发需求，本文档定义了标准化的AI通信协议，确保不同AI平台之间能够高效协作，减少误解和冲突。

1. 多AI平台协调架构

1.1 核心平台分工

根据项目分析，推荐以下AI平台分工架构：

- ChatGPT 5.2: 总指挥和架构师
 - 负责整体规划和标准制定
 - 协调各AI平台工作流程
 - 制定技术决策和架构方向
- Kiro Code: 主框架搭建
 - 负责后端架构设计
 - 数据库模型定义
 - API接口规范制定
- Antigravity: 前端开发
 - React组件开发
 - UI/UX设计实现
 - 前端状态管理

- Claude Code: 后端实现
 - 业务逻辑编写
 - API接口实现
 - 数据库操作层开发
- CodeRabbit: 代码审查
 - 自动化代码质量检查
 - 安全漏洞检测
 - 性能优化建议
- GLM: 测试和质量保证
 - 单元测试编写
 - 集成测试执行
 - 测试覆盖率监控

1.2 通信协议标准

任务分配协议

- 使用Vibe Kanban作为中央协调平台
- 每个任务必须明确指定负责的AI平台
- 任务状态实时同步：待办、进行中、审查中、已完成

数据交换格式

- 统一使用JSON格式进行数据交换
- 遵循RESTful API设计原则
- 标准HTTP状态码和错误处理

版本控制协议

- 所有代码变更通过GitHub管理
- 分支命名规范: feature/platform-name/task-description
- 合并请求必须经过CodeRabbit审查

2. 并行处理能力优化

2.1 任务并行化策略

多AI平台协调开发的核心优势在于并行处理能力:

- 模块并行开发: 不同模块可以同时开发, 缩短整体项目周期
- 前后端并行: 前端和后端工作可以并行进行, 减少等待时间
- 测试并行执行: 在开发过程中同步进行测试, 及早发现问题

2.2 协调机制

- 任务编排: 通过Vibe Kanban进行可视化任务管理
- 依赖管理: 明确任务间的依赖关系, 合理安排执行顺序
- 冲突解决: 建立标准的冲突解决流程和升级机制

3. 质量保障机制

3.1 多重验证体系

- 交叉验证: 多个AI平台对关键代码进行交叉审查
- 自动化检测: CodeRabbit进行自动化代码质量检查
- 人工复核: 关键决策点需要人工确认

3.2 反馈循环

- 实时反馈: 通过Vibe Kanban实现实时状态更新
- 定期评估: 定期评估各AI平台的协作效果

- 持续改进: 根据反馈结果持续优化协作流程

4. 实施建议

4.1 环境搭建

- 部署Vibe Kanban: 作为中央协调平台
- 配置GitHub: 设置标准的仓库结构和权限
- 设置AI平台MCP: 配置各AI平台的连接和认证
- 建立规范文档: 制定详细的协作规范和标准

4.2 工作流程设计

- 创建项目看板: 在Vibe Kanban中创建项目管理看板
- 设置任务模板: 为不同类型的任务创建标准模板
- 配置AI平台集成: 将各AI平台集成到工作流中
- 建立审查流程: 设置代码审查和测试流程

结论

标准化AI通信协议是多AI平台协调开发成功的关键。通过建立清晰的分工、统一的通信标准和有效的协调机制，可以充分发挥各AI平台的专业优势，实现高效的协同开发。

2. 技术规范执行强化.md

概述

基于Mpango ERP项目的技术契约和开发规范，本文档详细说明如何通过多AI平台协调来强化技术规范的执行，确保代码质量和项目一致性。

1. 编程风格契约强化

1.1 后端技术规范

根据 kiro_coding style contract，后端开发必须严格遵循以下规范：

Python技术栈

- Python版本: Python 3.11+
- 代码格式化: Black (统一代码风格)
- 代码检查: Ruff (快速Python linter)
- 类型检查: Mypy (静态类型检查)

代码质量要求

- 所有函数必须包含类型注解
- 遵循PEP 8编码规范
- 强制使用Docstring文档
- 导入语句按标准顺序排列

配置文件要求

```
[tool.black]
line-length = 88
target-version = ['py311']

[tool.ruff]
select = ["E", "F", "W", "C90", "I", "N", "UP", "YTT", "S", "BLE", "FBT", "E"]
ignore = ["E501", "COM812", "ISC001"]
```

1.2 前端技术规范

根据 kiro_frontend_contract，前端开发规范包括：

技术栈要求

- React: 18+ (函数组件 + Hooks)
- 构建工具: Vite (快速开发和构建)
- 类型系统: TypeScript (严格类型检查)
- 样式框架: TailwindCSS (实用优先的CSS框架)
- 状态管理: Zustand (轻量级状态管理)

代码规范工具

- Prettier: 代码格式化
- ESLint: 代码质量检查
- TypeScript: 严格类型检查

组件结构规范

```
src/
├── components/           # 可复用组件
│   ├── atoms/           # 原子组件
│   ├── molecules/       # 分子组件
│   └── organisms/       # 有机体组件
├── pages/               # 页面组件
├── hooks/               # 自定义Hooks
├── stores/              # Zustand状态管理
├── types/               # TypeScript类型定义
└── utils/               # 工具函数
```

2. AI辅助开发工具集成

2.1 Amazon CodeWhisperer集成

根据编程契约，鼓励使用Amazon CodeWhisperer提升开发效率：

- 代码生成: 基于注释和上下文生成代码

- 最佳实践: 自动应用行业最佳实践
- 安全检查: 识别潜在的安全问题

2.2 多AI平台协作强化

Kiro Code职责强化

- 严格按照FastAPI目录规范组织后端代码
- 确保数据库模型符合PostgreSQL 15+规范
- 遵循SQLAlchemy 2.0+和Alembic迁移管理标准

Claude Code执行强化

- 所有业务逻辑必须包含完整的错误处理
- API接口严格遵循RESTful设计原则
- 数据库操作必须使用事务管理

CodeRabbit审查强化

- 自动检查代码是否符合Black和Ruff规范
- 验证TypeScript类型定义的完整性
- 检查安全漏洞和性能问题

3. CI/CD集成强化

3.1 GitHub Actions workflow

根据 `kiro_ci_cd_contract` , 建立完整的CI/CD流程:

```
name: CI/CD Pipeline
```

```
on:
```

```
  push:
```

```

    branches: [ main, develop ]
pull_request:
    branches: [ main ]

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
      - name: Run Black
        run: black --check .
      - name: Run Ruff
        run: ruff check .
      - name: Run Mypy
        run: mypy .
      - name: Run tests
        run: pytest --cov=./ --cov-report=xml

  frontend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      - name: Install dependencies
        run: npm ci
      - name: Run ESLint
        run: npm run lint
      - name: Run TypeScript check
        run: npm run type-check
      - name: Run tests
        run: npm run test

```

3.2 质量门槛设置

- 代码覆盖率: 后端≥80%，前端≥70%

- 类型检查: TypeScript严格模式, 无any类型
- 安全检查: 通过CodeRabbit安全扫描
- 性能检查: 构建时间和包大小限制

4. 数据库规范强化

4.1 PostgreSQL规范

根据 `kiro_database_contract` :

命名约定

- 表名: snake_case, 复数形式
- 字段名: snake_case
- 索引名: idx_table_column
- 外键名: fk_table_column

核心模型规范

```
# 用户模型示例
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    created_at: Mapped[datetime] = mapped_column(default=func.now())
    updated_at: Mapped[datetime] = mapped_column(default=func.now(), onupdate=func.now())
```

4.2 迁移管理

- 使用Alembic进行数据库迁移
- 所有迁移必须可逆
- 迁移脚本必须包含详细注释

5. 测试规范强化

5.1 测试结构

根据 kiro_test_contract :

```
tests/
├── unit/           # 单元测试
├── integration/    # 集成测试
├── api/            # API测试
├── fixtures/       # 测试数据
└── conftest.py     # pytest配置
```

5.2 测试要求

- 单元测试: 覆盖所有业务逻辑函数
- 集成测试: 测试组件间交互
- API测试: 使用pytest + httpx测试所有API端点
- 前端测试: 使用Jest + React Testing Library

6. Docker化规范

6.1 容器配置

根据 kiro_docker_contract :

```
# 后端Dockerfile示例
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

6.2 开发环境

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mpango
    depends_on:
      - db

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    volumes:
      - ./frontend:/app

  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mpango
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
```

结论

通过多AI平台的协调配合，可以有效强化技术规范的执行。每个AI平台在其专业领域内确保规范的严格遵循，通过自动化工具和人工审查相结合的方式，建立完善的质量保障体系。

3. 安全与权限控制.md

概述

基于Mpango ERP项目的安全需求和RBAC权限矩阵，本文档详细说明多AI平台协调开发中的安全措施和权限控制机制，确保系统安全性和数据保护。

1. 基于角色的访问控制(RBAC)

1.1 RBAC矩阵实现

根据 RBAC Matrix (MVP)，系统定义了以下核心角色：

管理员角色(Admin)

- 用户管理: 创建、编辑、删除用户账户
- 系统配置: 修改系统设置和参数
- 数据访问: 访问所有业务数据
- 报表权限: 生成和查看所有报表

销售角色(Sales)

- 客户管理: 管理客户信息和关系
- 订单处理: 创建和处理销售订单
- 价格查询: 查看产品价格和库存
- 销售报表: 查看销售相关报表

仓库角色(Warehouse)

- 库存管理: 管理产品库存和入出库
- 订单履行: 处理订单发货和配送
- 库存报表: 查看库存相关报表
- 供应商协调: 与供应商进行库存协调

财务角色(Finance)

- 财务记录: 管理财务交易和记录
- 发票管理: 创建和管理发票
- 支付处理: 处理付款和收款
- 财务报表: 生成和查看财务报表

1.2 权限实现机制

```
# 权限装饰器示例
from functools import wraps
from fastapi import HTTPException, Depends
from sqlalchemy.orm import Session

def require_permission(permission: str):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            current_user = kwargs.get('current_user')
            if not current_user:
                raise HTTPException(status_code=401, detail="未认证")

            if not has_permission(current_user, permission):
                raise HTTPException(status_code=403, detail="权限不足")

            return await func(*args, **kwargs)
        return wrapper
    return decorator

# 使用示例
@app.get("/admin/users")
@require_permission("user.read")
async def get_users(current_user: User = Depends(get_current_user)):
    return {"users": []}
```

2. 多租户安全架构

2.1 租户隔离策略

根据 Multi-Tenancy Spec (MVP)，采用每个租户一个模式的策略：

数据隔离

- 模式级隔离: 每个租户使用独立的数据库模式
- 连接池管理: 为每个租户维护独立的连接池
- 数据访问控制: 确保租户间数据完全隔离

```
# 租户中间件示例
class TenantMiddleware:
    def __init__(self, app):
        self.app = app

    async def __call__(self, scope, receive, send):
        if scope["type"] == "http":
            # 从请求头或域名提取租户信息
            tenant_id = extract_tenant_id(scope)
            if tenant_id:
                # 设置数据库模式
                set_search_path(tenant_id)

        await self.app(scope, receive, send)
```

安全边界

- API隔离: 租户间API调用完全隔离
- 文件存储: 租户文件存储在独立目录
- 缓存隔离: Redis缓存按租户前缀隔离

2.2 租户认证机制

```
# JWT令牌包含租户信息
def create_access_token(user_id: int, tenant_id: str):
    payload = {
        "user_id": user_id,
        "tenant_id": tenant_id,
        "exp": datetime.utcnow() + timedelta(hours=24)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

3. API安全规范

3.1 认证与授权

根据 `kiro_api_contract`，API安全包括：

JWT认证

- 令牌生成: 登录成功后生成JWT令牌
- 令牌验证: 每个API请求验证令牌有效性
- 令牌刷新: 提供令牌刷新机制

```
# API认证示例
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

security = HTTPBearer()

async def get_current_user(token: str = Depends(security)):
    try:
        payload = jwt.decode(token.credentials, SECRET_KEY, algorithms=["HS256"])
        user_id = payload.get("user_id")
        tenant_id = payload.get("tenant_id")

        if not user_id or not tenant_id:
            raise HTTPException(status_code=401, detail="无效令牌")

        # 设置租户上下文
        set_tenant_context(tenant_id)

        return get_user_by_id(user_id)
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="令牌已过期")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="无效令牌")
```

API限流

- 请求频率限制: 防止API滥用

- IP白名单: 限制特定IP访问
- 用户级限流: 按用户限制请求频率

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@app.get("/api/users")
@limiter.limit("100/minute")
async def get_users(request: Request):
    return {"users": []}
```

3.2 数据验证与清理

输入验证

```
from pydantic import BaseModel, validator

class UserCreateRequest(BaseModel):
    email: str
    password: str
    role: str

    @validator('email')
    def validate_email(cls, v):
        if '@' not in v:
            raise ValueError('无效邮箱格式')
        return v

    @validator('password')
    def validate_password(cls, v):
        if len(v) < 8:
            raise ValueError('密码长度至少8位')
        return v
```

SQL注入防护

- 使用SQLAlchemy ORM防止SQL注入

- 参数化查询
- 输入转义和验证

4. 数据安全性与加密

4.1 敏感数据加密

密码加密

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

数据库字段加密

```
from cryptography.fernet import Fernet

class EncryptedField:
    def __init__(self, key: bytes):
        self.cipher = Fernet(key)

    def encrypt(self, data: str) -> str:
        return self.cipher.encrypt(data.encode()).decode()

    def decrypt(self, encrypted_data: str) -> str:
        return self.cipher.decrypt(encrypted_data.encode()).decode()
```

4.2 传输安全

HTTPS强制

- 所有API通信强制使用HTTPS
- HTTP请求自动重定向到HTTPS
- HSTS头部设置

```
from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware

app.add_middleware(HTTPSRedirectMiddleware)
app.add_middleware(TrustedHostMiddleware, allowed_hosts=["*.example.com"])
```

5. 审计日志与监控

5.1 操作审计

审计日志记录

```
class AuditLog(Base):
    __tablename__ = "audit_logs"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    tenant_id: Mapped[str] = mapped_column(String(50))
    action: Mapped[str] = mapped_column(String(100))
    resource: Mapped[str] = mapped_column(String(100))
    timestamp: Mapped[datetime] = mapped_column(default=func.now())
    ip_address: Mapped[str] = mapped_column(String(45))
    user_agent: Mapped[str] = mapped_column(Text)

def log_action(user_id: int, action: str, resource: str, request: Request):
    audit_log = AuditLog(
        user_id=user_id,
        tenant_id=get_current_tenant(),
        action=action,
        resource=resource,
        ip_address=request.client.host,
        user_agent=request.headers.get("user-agent")
    )
    db.add(audit_log)
    db.commit()
```

5.2 安全监控

异常检测

- 登录失败次数监控
- 异常API访问模式检测
- 数据访问异常监控

```
class SecurityMonitor:
    def __init__(self):
        self.failed_attempts = {}

    def record_failed_login(self, ip_address: str):
        if ip_address not in self.failed_attempts:
            self.failed_attempts[ip_address] = 0

        self.failed_attempts[ip_address] += 1

        if self.failed_attempts[ip_address] > 5:
            # 触发安全警报
            self.trigger_security_alert(ip_address)

    def trigger_security_alert(self, ip_address: str):
        # 发送安全警报
        logger.warning(f"检测到可疑登录活动: {ip_address}")
```

6. AI平台安全协调

6.1 AI平台权限管理

平台访问控制

- Kiro Code: 仅允许访问架构和配置文件
- Claude Code: 限制在业务逻辑代码范围内
- CodeRabbit: 只读权限，用于代码审查
- GLM: 测试环境访问权限

API密钥管理

```
class AIServiceAuth:
    def __init__(self):
        self.api_keys = {
            "kiro": os.getenv("KIRO_API_KEY"),
            "claude": os.getenv("CLAUDE_API_KEY"),
            "coderabbit": os.getenv("CODERABBIT_API_KEY"),
            "glm": os.getenv("GLM_API_KEY")
        }

    def get_auth_header(self, service: str) -> dict:
        if service not in self.api_keys:
            raise ValueError(f"未知服务: {service}")

        return {"Authorization": f"Bearer {self.api_keys[service]}"}

```

6.2 代码安全审查

自动化安全检查

- 静态代码分析: 使用CodeRabbit检测安全漏洞
- 依赖安全扫描: 检查第三方库安全性
- 敏感信息检测: 防止密钥和敏感信息泄露

```
# 敏感信息检测规则
SENSITIVE_PATTERNS = [
    r'password\s*=\s*["'](?:\^'|')+',
    r'api_key\s*=\s*["'](?:\^'|')+',
    r'secret\s*=\s*["'](?:\^'|')+',
]

def check_sensitive_info(code: str) -> List[str]:
    violations = []
    for pattern in SENSITIVE_PATTERNS:
        if re.search(pattern, code, re.IGNORECASE):
            violations.append(f"检测到敏感信息: {pattern}")
    return violations

```

7. 环境安全配置

7.1 生产环境安全

环境变量管理

```
# .env.production
DATABASE_URL=postgresql://user:password@localhost:5432/mpango_prod
SECRET_KEY=your-secret-key-here
JWT_SECRET=your-jwt-secret-here
REDIS_URL=redis://localhost:6379/0

# 加密敏感配置
ENCRYPTION_KEY=your-encryption-key-here
```

Docker安全配置

```
# 使用非root用户运行
FROM python:3.11-slim

RUN groupadd -r appuser && useradd -r -g appuser appuser

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN chown -R appuser:appuser /app

USER appuser

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

7.2 网络安全

防火墙配置

- 仅开放必要端口(80, 443, 22)

- 限制数据库端口访问
- 配置VPC和安全组

SSL/TLS配置

```
server {  
    listen 443 ssl http2;  
    server_name api.example.com;  
  
    ssl_certificate /path/to/certificate.crt;  
    ssl_certificate_key /path/to/private.key;  
  
    ssl_protocols TLSv1.2 TLSv1.3;  
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512;  
  
    location / {  
        proxy_pass http://backend:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

结论

通过建立完善的安全与权限控制体系，多AI平台协调开发可以在保证开发效率的同时，确保系统的安全性和数据保护。关键在于建立清晰的权限边界、实施有效的安全监控，以及确保各AI平台在其授权范围内安全运行。